Models and Tools for Cyber-Physical Systems

Chapter 4: Asynchronous Model

Instructors:Marco Muñiz
{muniz}@cs.aau.dk

Slides courtesy of Rajeev Alur
alur@cis.upenn.edu

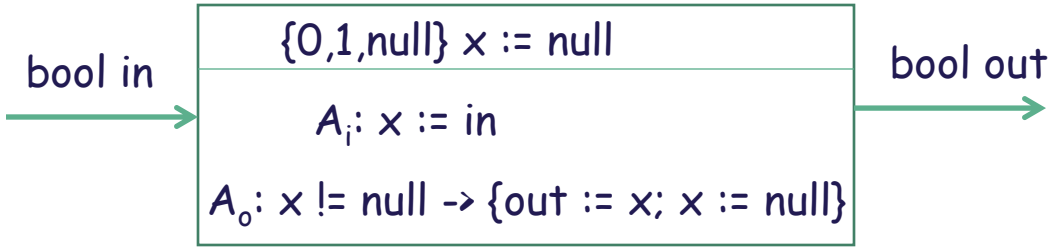
Asynchronous Models

- Recall: In a synchronous model, all components execute in a sequence of (logical) rounds in lock-step

- Asynchronous model: Speed at which different components execute is independent (or unknown)
  - Processes in a distributed system
  - Threads in a typical operating system

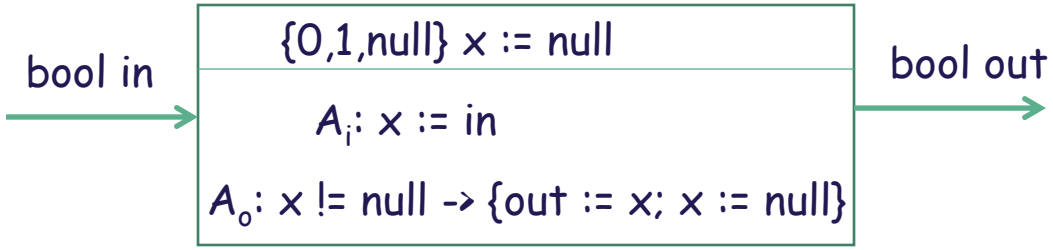
- Key design challenge: how to achieve coordination?

Example: Asynchronous Buffer

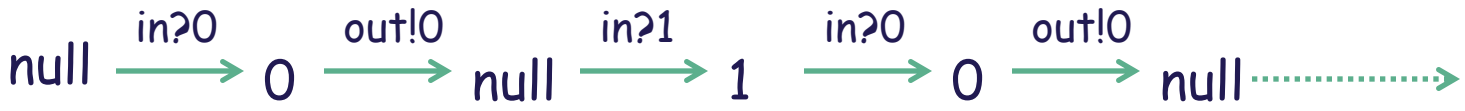


- Input channel: in of type Boolean
- Output channel: out of type Boolean
- State variable: x, can be null or hold 0/1 value
- Initialization of state variables: assignment x := null
- Input task A_i for processing of inputs: code x := in
- Output task A_o for producing outputs:
  guard: x != null update: out := x; x := null

Example: Asynchronous Buffer

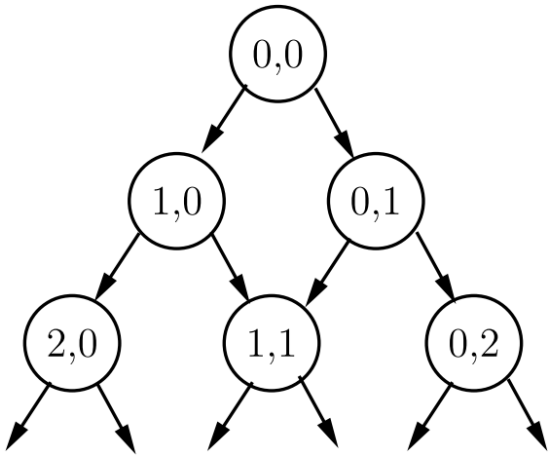


- Execution model: In one step, only a single task is executed
  - Processing of inputs (by input tasks) is decoupled from production of outputs (by output tasks)
- A task can be executed if it is enabled, i.e., its guard condition holds
  - If multiple tasks are enabled, only one of them is executed
- Sample execution:



Example: Asynchronous Increments

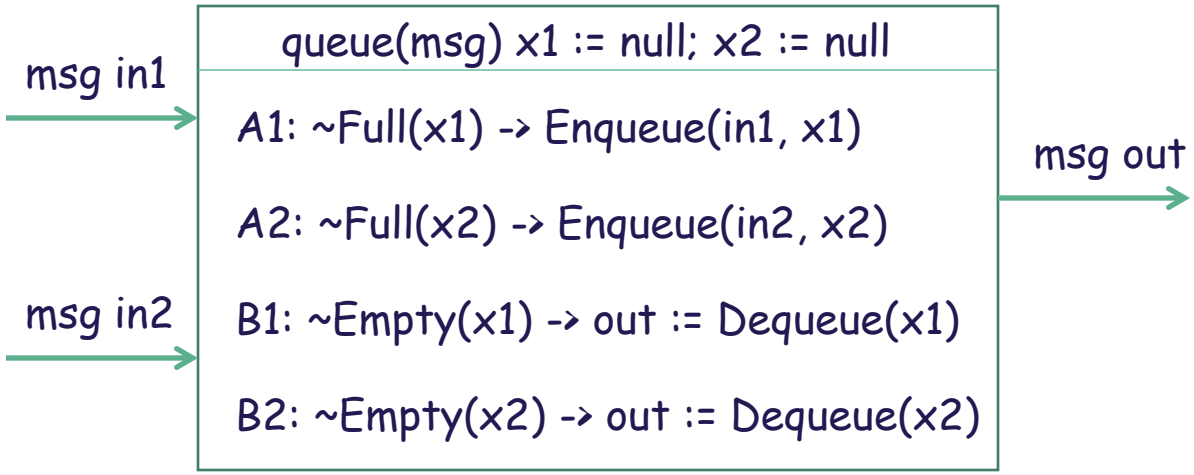
nat x := 0; y := 0
A_x: x := x+1
A_y: y := y+1



- Internal task: Does not involve input or output channels
  - Can have a guard condition (none in the example)
- In each step, execute either task A_x or task A_y
- Sample execution:

  (0,0) - ε -> (1,0) - ε -> (1,1) - ε -> (1,2) - ε -> (1,3) - ε -> …
  - ε -> (1,105) - ε -> (2,105) …
- For all natural numbers m, n, the state (m, n) is reachable
- Interleaving semantics

Asynchronous Merge



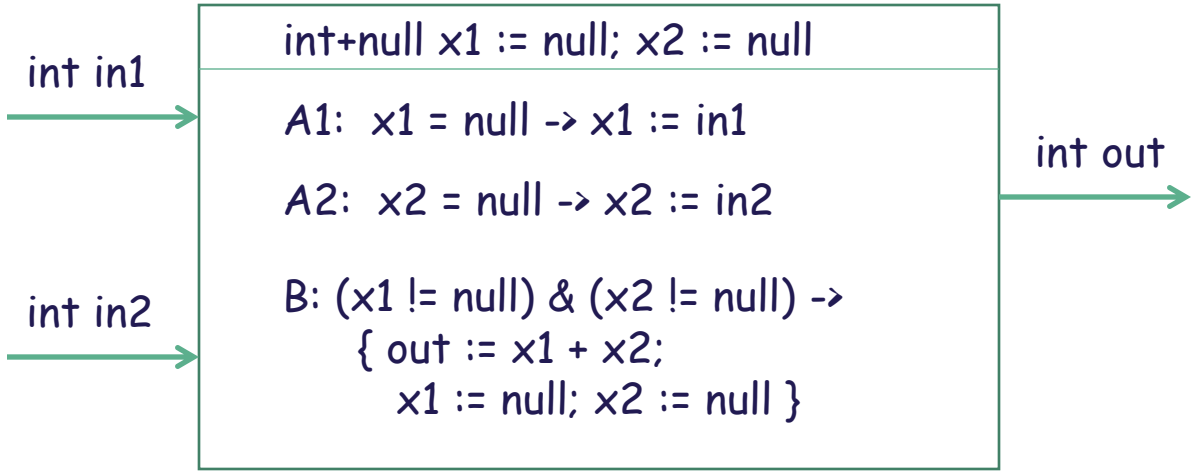
Sequence of messages on output channel is an arbitrary merge of sequences of values on the two input channels

At every step, exactly one of the four tasks executes (provided its guard condition holds)

Sample execution ("null" represents the empty list):

([],[]) – in1?5 -> ([5],[]) – in2?0 -> ([5],[0]) – out!0 -> ([5],[])
– in1?6 -> ([5,6],[]) – in2?3 -> ([5,6],[3]) – out!5 -> ([6],[3]) …

What does this process do?



Here "int+null" is a type like int but with the option to be null

Definition: Asynchronous Process (1)

- Set I of (typed) input channels
  - Defines the set of inputs of the form x?v, where x is an input channel and v is a value
- Set O of (typed) output channels
  - Defines the set of outputs of the form y!v, where y is an output channel and v is a value
- Set S of (typed) state variables
  - Defines the set of states Q_S
- Initialization Init
  - Defines the set [Init] of initial states

Definition: Asynchronous Process (2)

- Set of input tasks; each such task is associated with an input channel x
  - Guard condition over state variables S
  - Update code from read-set S U {x} to write-set S
  - Defines a set of input actions of the form s – x?v -> t

- Set of output tasks; each task is associated with an output channel y
  - Guard condition over state variables S
  - Update code from read-set S to write-set S U {y}
  - Defines a set of output actions of the form s – y!v -> t

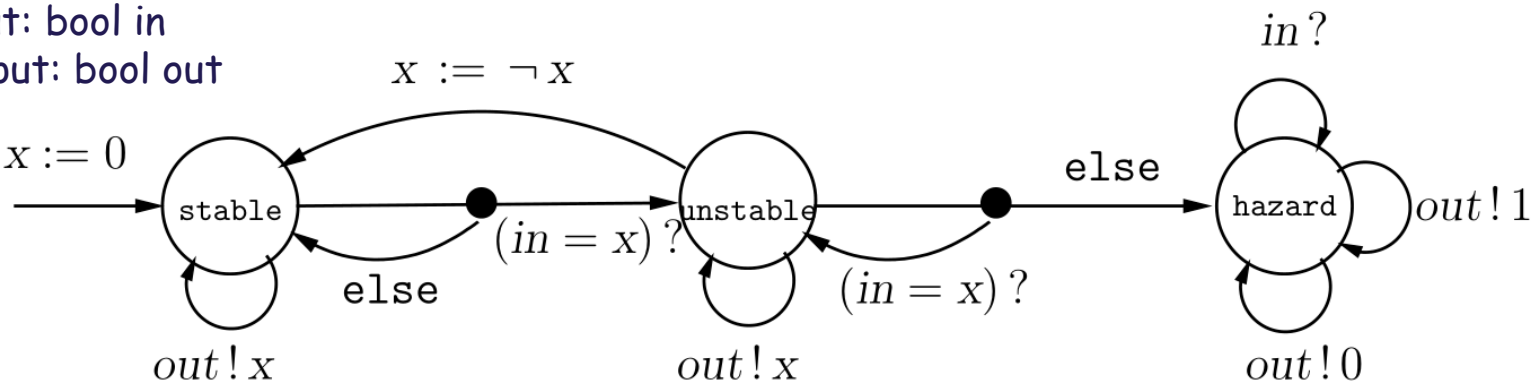
- Set of internal tasks
  - Guard condition over state variables S
  - Update code from read-set S to write-set S
  - Defines a set of internal actions of the form s – ε -> t

Asynchronous Gates and Circuits

- Why design asynchronous circuits?
  - Input can be changed even before the effect propagates through the entire circuit
  - Can be faster than synchronous circuits, but design is more complex
- Asynchronous NOT gate
  - When input changes, gate enters *unstable* state until it gets a chance to update the output value
  - If input changes again in unstable state, this causes a *hazard* where behavior is unpredictable

Asynchronous NOT Gate as Extended State Machine

Input: bool in
Output: bool out



Each mode-switch/transition corresponds to a task

- Example input task: (mode = stable) -> if (in = x) then mode := unstable
- Example output task: (mode = stable) -> out := x
- Example internal task: (mode = unstable) -> {x := ~x; mode := stable}

Sample execution:

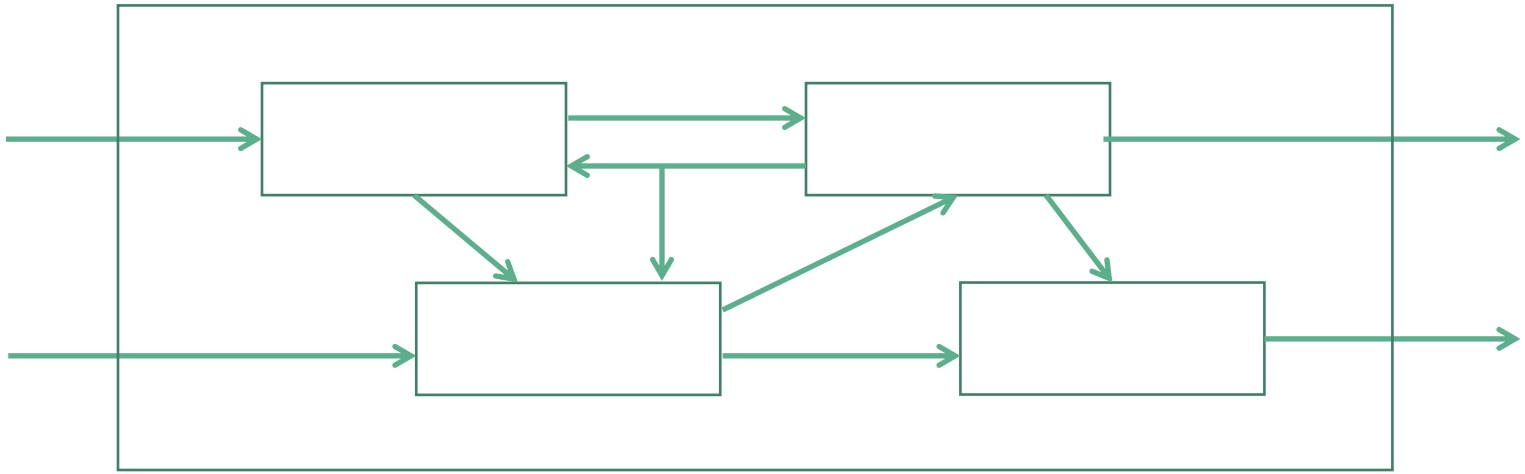
(stable,0) – in?1 -> (stable,0) – out!0 -> (stable,0) – in?0 -> (unstable,0) – ε -> (stable,1) – out!1 -> (stable,1) – in?1 -> (unstable,1) – out!1 -> (unstable,1) – in?0 -> (hazard,1) – out!0 -> (hazard,1) – out!1 -> (hazard,1) …

How can an environment ensure that the gate does not enter the hazard mode?

- When the input toggles, wait to observe a change in the value of the output

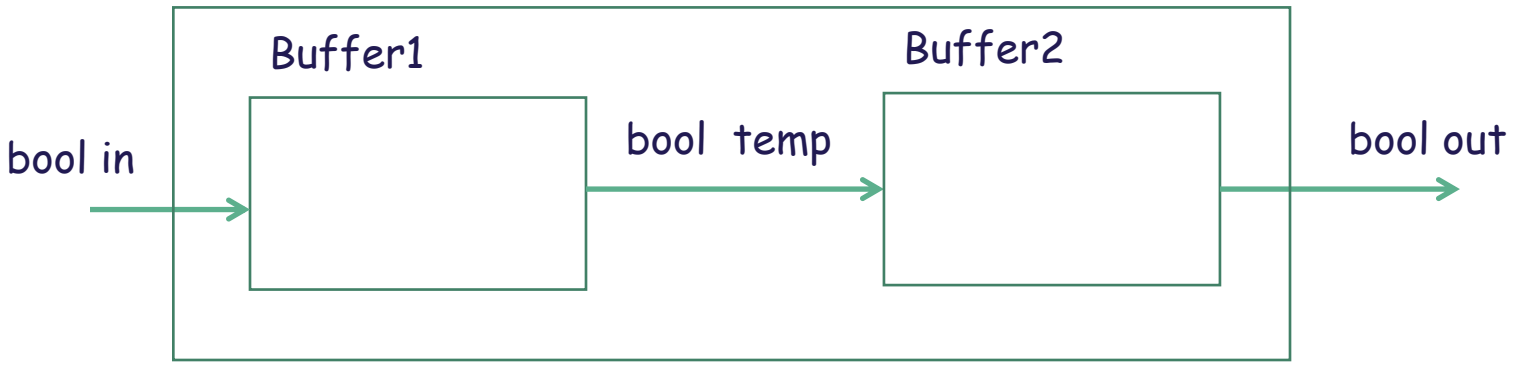
Composition

Block Diagrams



- Visually the same as in the synchronous case
  - Execution semantics different!

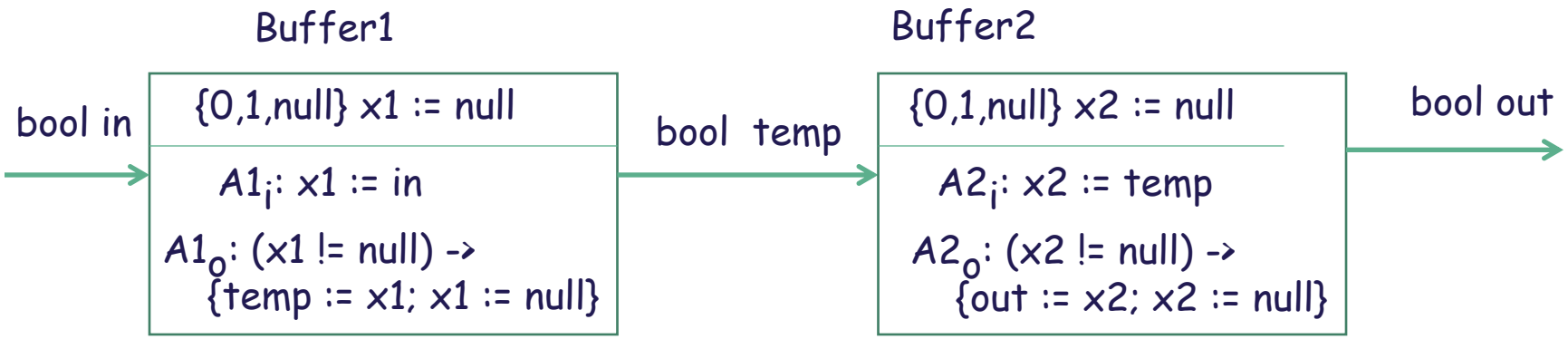
DoubleBuffer



(Buffer[out -> temp] | Buffer[in -> temp]) \ temp

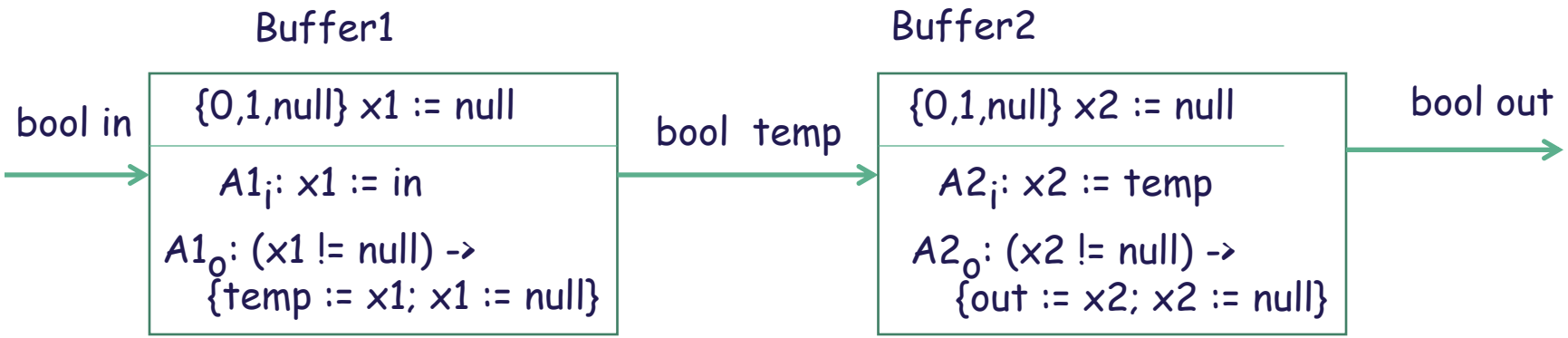
- Instantiation: Create two instances of Buffer
  - Output of Buffer1 = Input of Buffer2 = variable temp
- Parallel composition: Asynchronous concurrent execution of Buffer1's output and Buffer2's input (note: new syntax A | B)
- Hide variable temp: Encapsulation (temp becomes local)

Composing Buffer1 and Buffer2

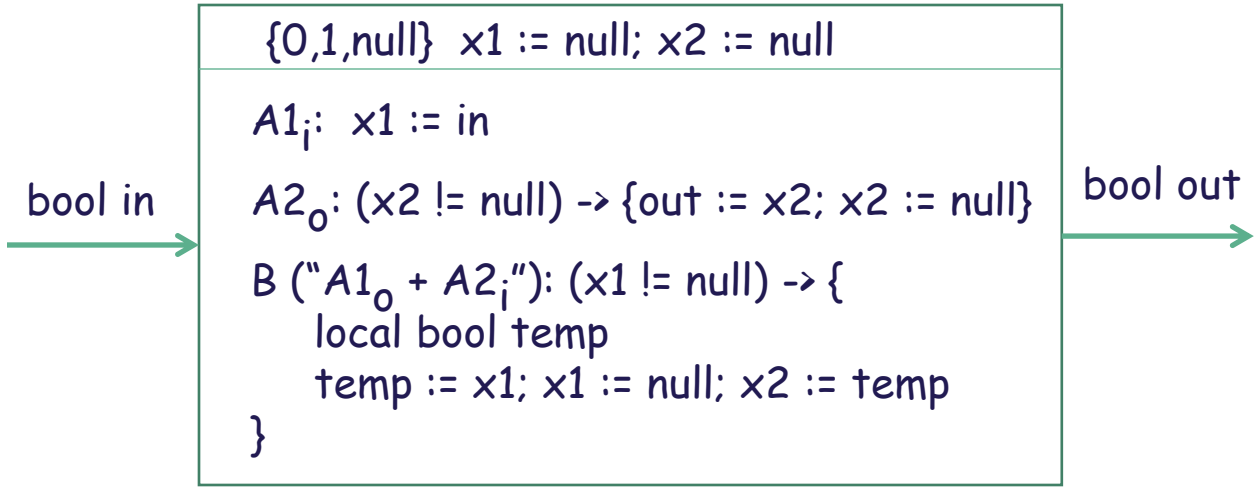


- Inputs, outputs, states, and initialization for composition obtained in the same manner as in synchronous case
- What are the tasks of the composition?
  - Production of output on channel temp by Buffer1 synchronized with consumption of input on channel temp by Buffer2

Compiled DoubleBuffer



(Buffer[out -> temp] | Buffer[in -> temp]) \ temp



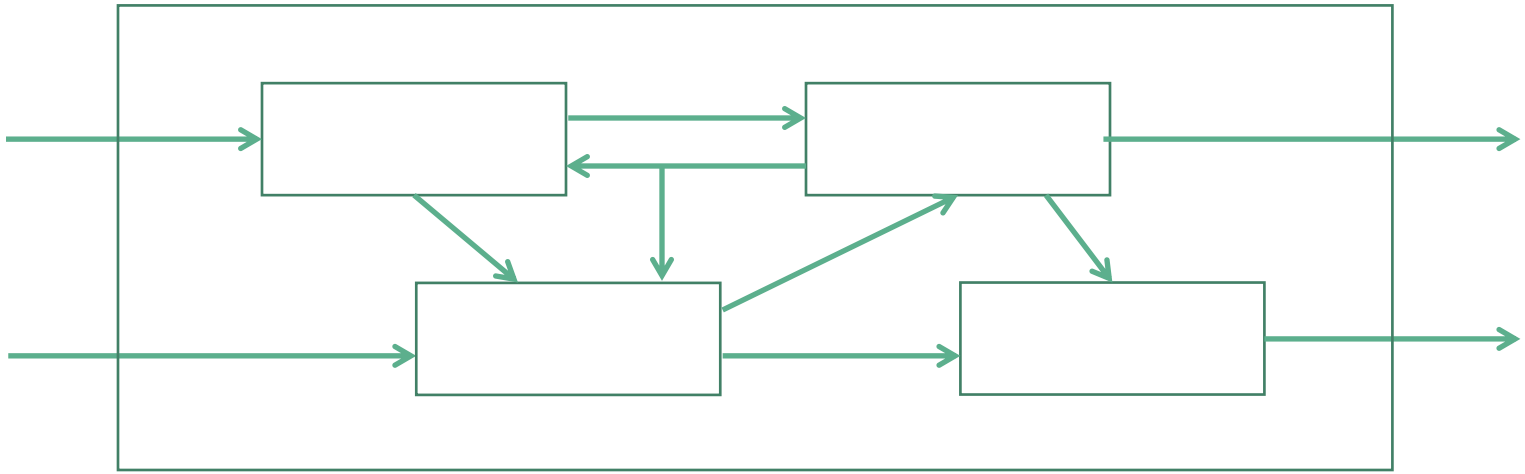
Definition of Asynchronous Composition

- Given asynchronous processes P1 and P2, how to define P1 | P2 ?

- Note: In each step of execution, only one task is executed
  - Concepts such as await-dependencies are not relevant

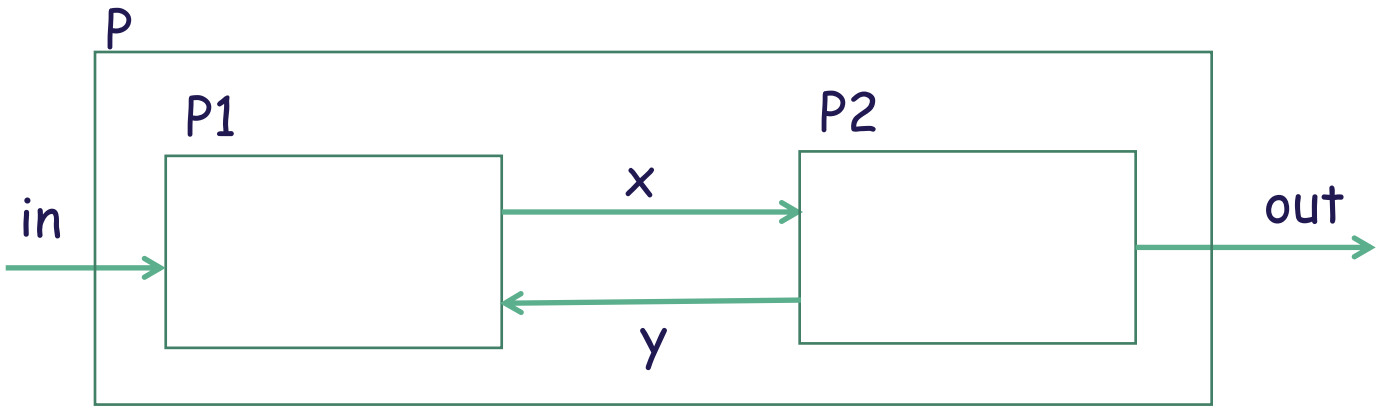
- Sample case (see textbook for complete definition):
  - If y is an output channel of P1 and input channel of P2, and
  - A1 is an output task of P1 for y with code: Guard1 -> Update1, and
  - A2 is an input task of P2 for y with code: Guard2 -> Update2, then
  - Composition has an output task for y with code:
    (Guard1 & Guard2) -> Update1; Update2

Execution Model: Another View



- A single step of execution
  - Execute an internal task of one of the processes
  - Process input on an external channel x: Execute an input task for x of every process to which x is an input
  - Execute an output task for an output y of some process, followed by an input task for y for every process to which y is an input
- If multiple enabled choices, choose one non-deterministically
  - No constraint on relative execution speeds

Asynchronous Execution



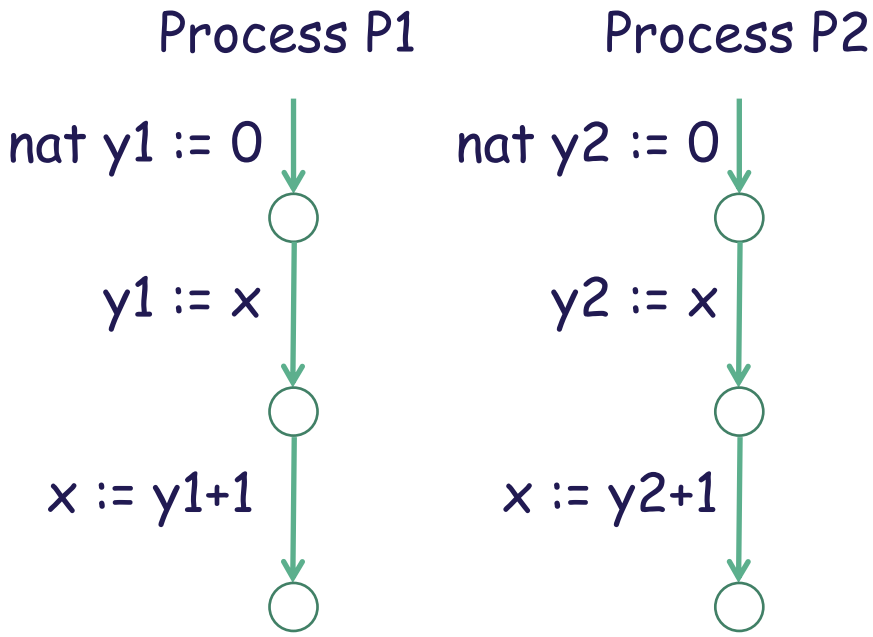
What can happen in a single step of this asynchronous model P?

- P1 synchronizes with the environment to accept input on in
- P2 synchronizes with the environment to send output on out
- P1 performs some internal computation (one of its internal tasks)
- P2 performs some internal computation (one of its internal tasks)
- P1 produces output on channel x, followed by its consumption by P2
- P2 produces output on channel y, followed by its consumption by P1

Shared Memory

Shared Memory Programs

AtomicReg nat x := 0



Declaration of shared variables
+ code for each process

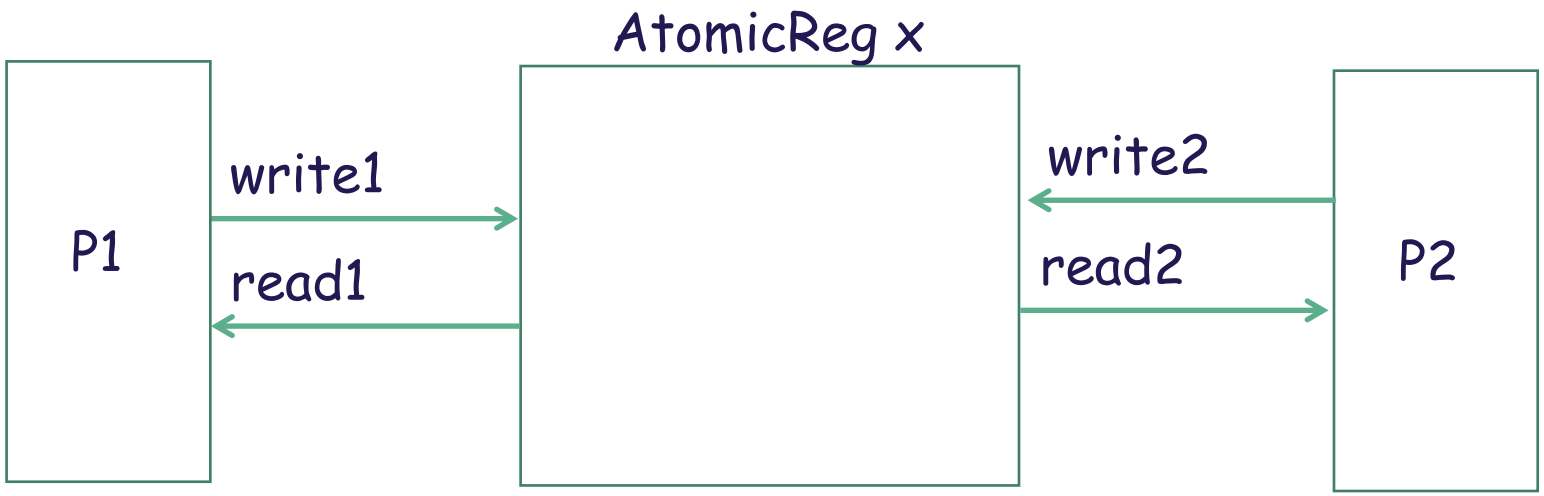
Key restriction: Each statement of a process either

- changes local variables,
- reads a single shared variable, or
- writes a single shared variable

Execution model: execute one step of one of the processes

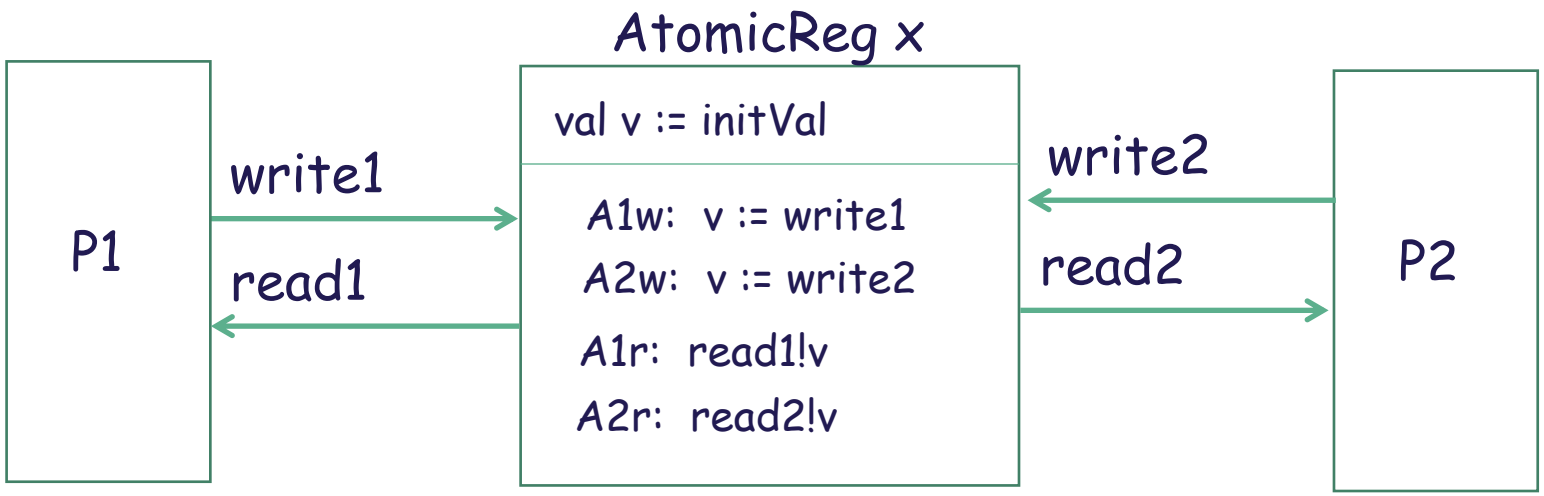
Can be formalized as asynchronous processes

Shared Memory Processes



- Processes P1 and P2 communicate by reading/writing shared variables
- Each shared variable can be modeled as an asynchronous process
  - State of each such process is the value of the corresponding variable
  - In implementation, shared memory can be a separate subsystem
- Read and write channel between each process and each shared variable
  - To write x, P1 synchronizes with x on "write1" channel
  - To read x, P2 synchronizes with x on "read2" channel

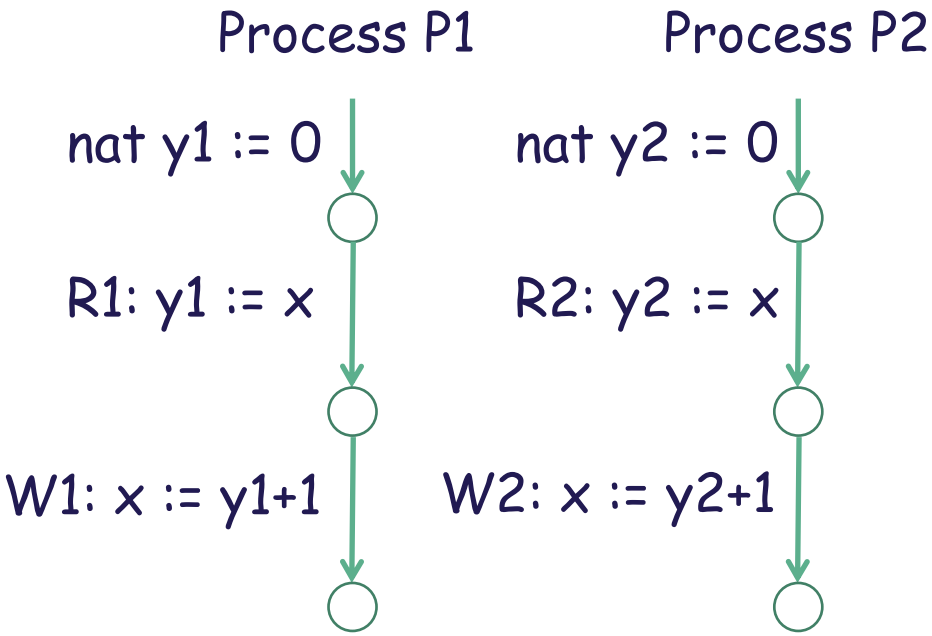
Atomic Registers



- By definition of our asynchronous model, each step above is either internal to P1 or P2, or involves exactly one synchronization: either read or write one shared variable by one of the processes
- Atomic register: Basic primitives are read and write
  - To "increment" such a register, a process first needs to read and then write back the incremented value
  - But these two are separate steps, and the register value can be changed in between by another process

Data Races

AtomicReg nat x := 0



What are the possible values of x after all steps are executed?

x can be 1 or 2

Possible executions:

- R1, R2, W1, W2
- R1, W1, R2, W2
- R1, R2, W2, W1
- R2, R1, W1, W2,
- R2, W2, R1, W1
- R2, R1, W2, W1

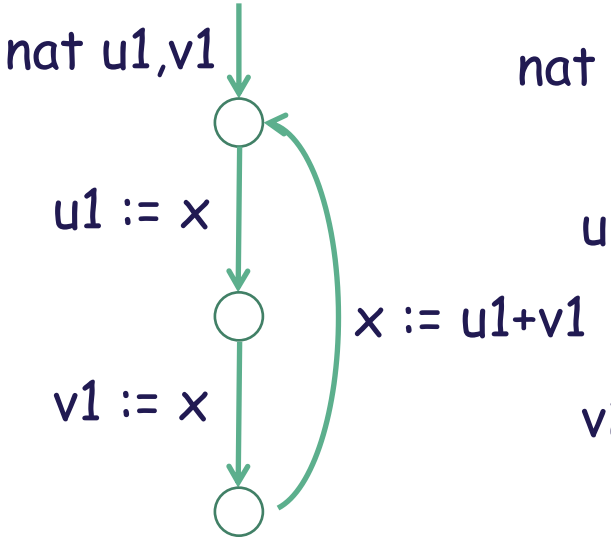
Data race: Concurrent accesses to a shared object where the result depends on the order of execution

This should be avoided!

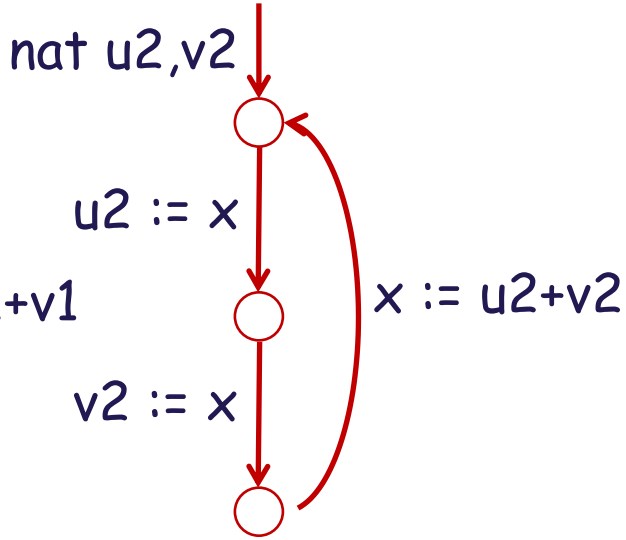
Puzzle

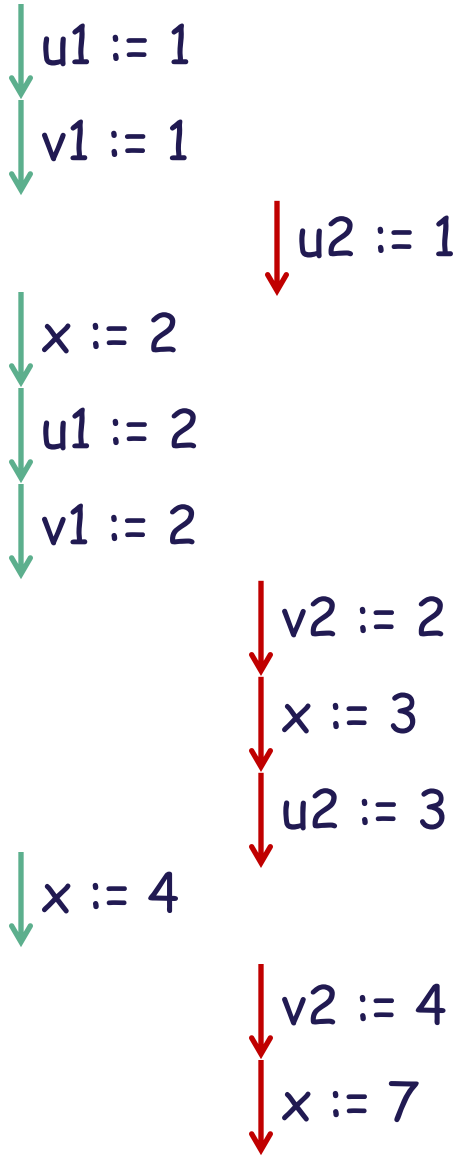
AtomicReg nat x := 1

Process P1



Process P2

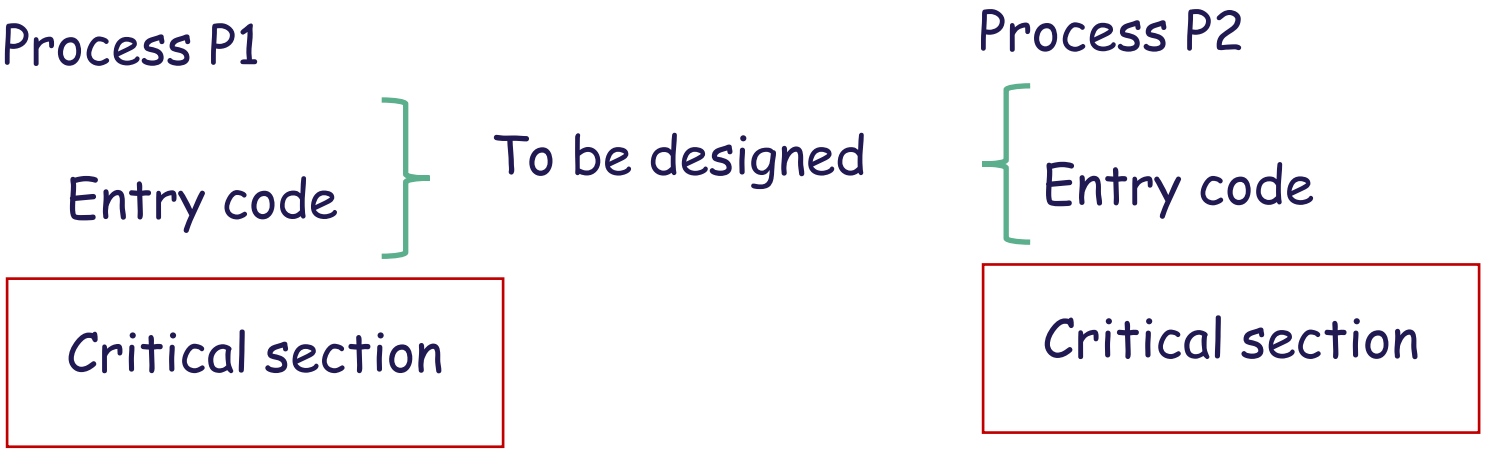




Which values can x have?

Every number > 0!

Mutual Exclusion Problem



- Critical section: Part of code that an asynchronous process should execute without interference from others
  - Critical section can include code to update shared objects/database
- Mutual exclusion problem: Design code to be executed before entering critical section by each process
  - Coordination using shared atomic registers
  - No assumption about how long a process stays in critical section
  - A process may want to enter critical section repeatedly

Requirements

Process P1

Entry code

Critical section

To be designed

Process P2

Entry code

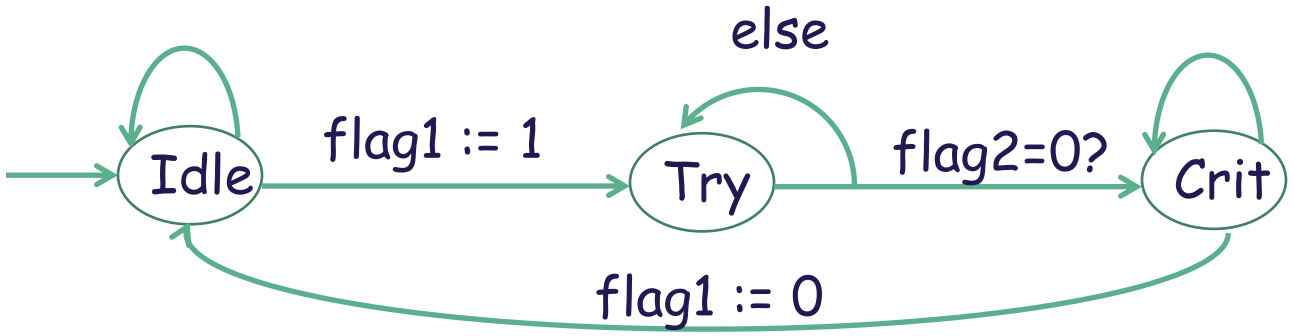
Critical section

- Safety requirement: Mutual exclusion: Both processes should not be in the critical section simultaneously (can be formalized as an invariant)
- Liveness requirement: Starvation freedom: If a process is trying to enter, then eventually it should be able to enter

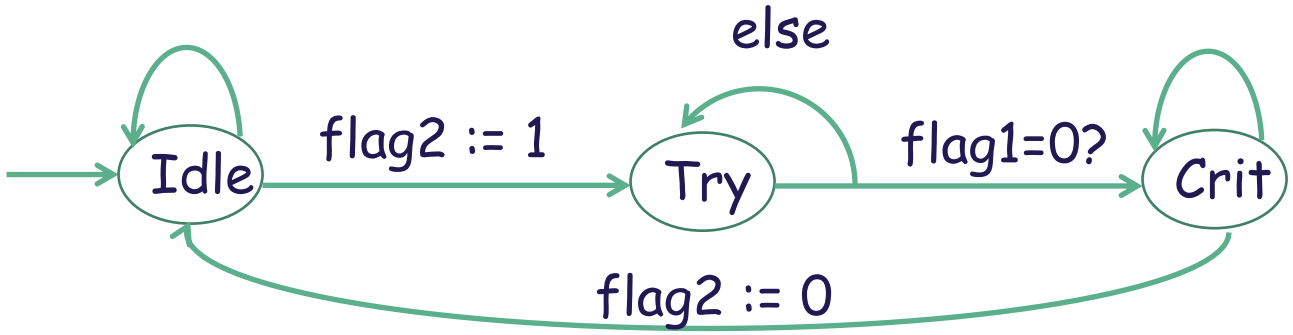
Mutual Exclusion: First Attempt

AtomicReg bool flag1 := 0; flag2 := 0

Process P1



Process P2

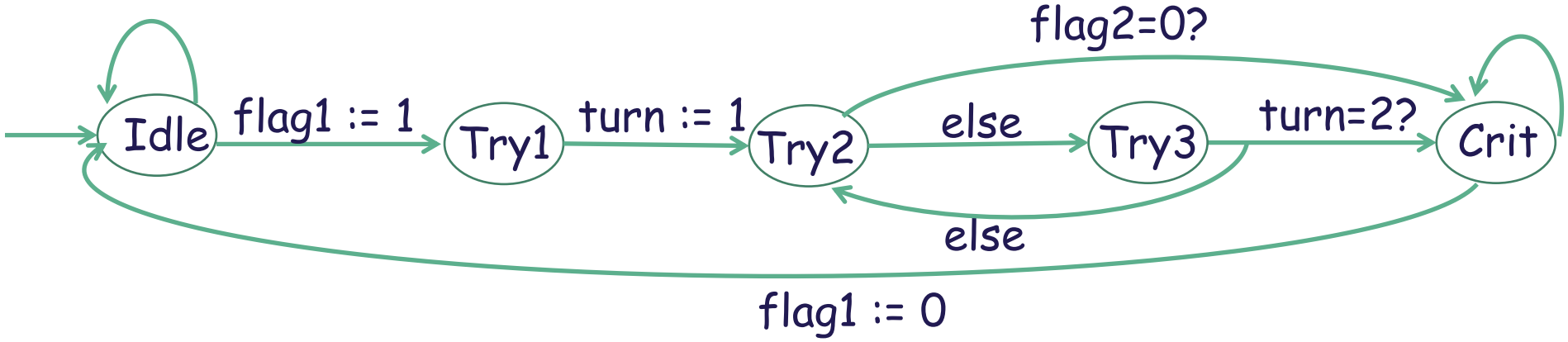


Not starvation-free!
(see Uppaal model)

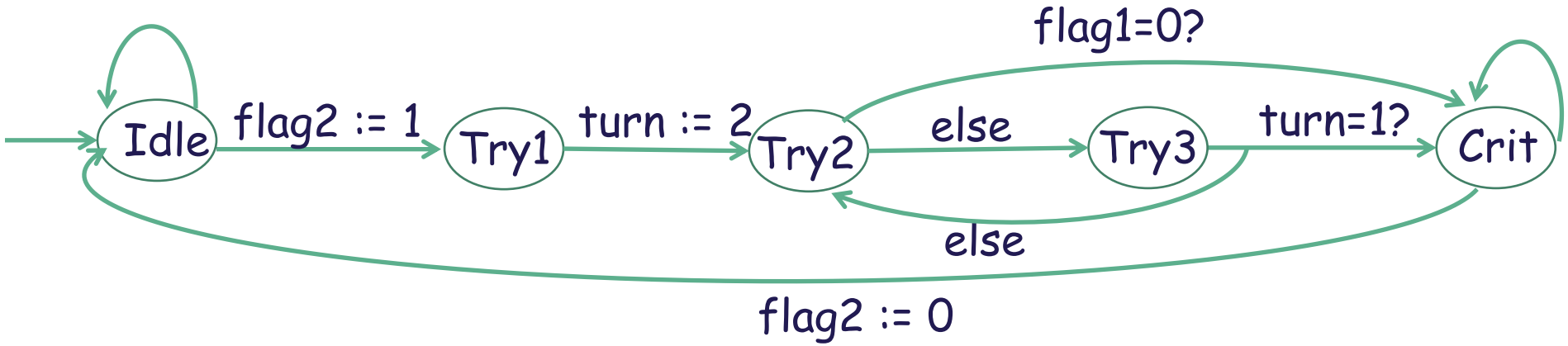
Peterson's Mutual Exclusion Protocol

AtomicReg bool flag1 := 0; flag2 := 0; {1,2} turn

Process P1



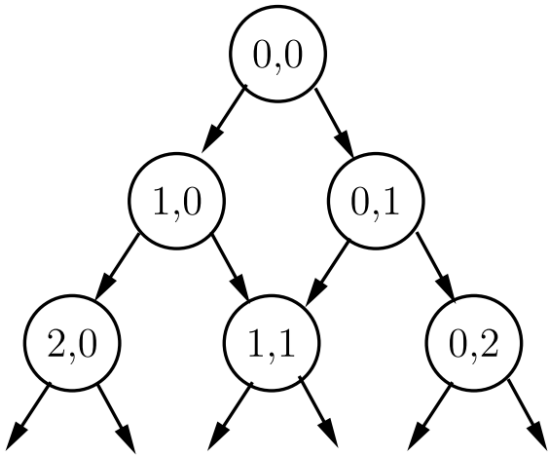
Process P2



Fairness

Another Look at Asynchronous Execution Model

nat x := 0; y := 0
A_x: x := x+1
A_y: y := y+1



- Tasks A_x and A_y execute in an arbitrary order
  - Motivation: If we establish that all possible executions of this asynchronous design satisfy some requirement, then this requirement will hold in every implementation
- Are the following realistic executions?

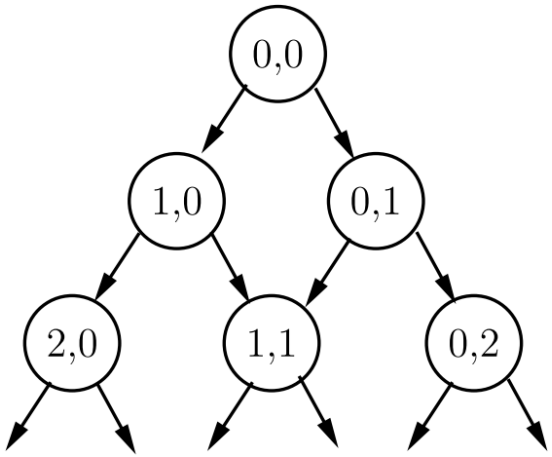
  (0,0) -A_x-> (1,0) -A_x-> (2,0) -A_x-> (3,0) … -A_x-> (105,0) -A_x-> …

  (0,0) -A_x-> (1,0) -A_x-> (2,0) -A_y-> (2,1) -A_y-> (2,2) … -A_y-> (2,105) -A_y-> …
- Does the system satisfy the following requirement?

  "In every execution, values of both x and y eventually exceed 10"

Fairness Assumption

nat x := 0; y := 0
A_x: x := x+1
A_y: y := y+1



- Fairness assumption for a task
  - Assumption about the underlying platform/scheduler
  - Informally, an infinite execution is unfair for a task if the task does not get a chance to execute
- Unfair to A_y: (0,0) -A_x-> (1,0) -A_x-> (2,0) -A_x-> (3,0) … -A_x-> (105,0) …
- Unfair to A_x: (0,0) -A_x-> (1,0) -A_x-> (2,0) -A_y-> (2,1) -A_y-> (2,2) … (2,105) …
- Fairness assumptions restrict the set of "possible" executions to realistic ones without putting concrete bounds on relative speeds

Formalizing Fairness

nat x := 0; y := 0
A_x: x := x+1
A_y: x = 0 -> y := y+1

- **Definition 1 (Unconditional fairness)**: An infinite execution is *unconditionally fair* to a task A if task A is executed repeatedly (i.e., infinitely often) during this execution
- Is the following execution unconditionally fair to task A_y?

  (0,0) -A_x-> (1,0) -A_x-> (2,0) -A_x-> (3,0) -A_x-> … (105,0) -A_x-> …
- No, but one would intuitively call it fair because, after the first step, task A_y is not enabled anymore and thus cannot be executed
- **Definition 2 (Weak fairness)**: An infinite execution is *weakly fair* to a task A if, repeatedly, task A is either executed or disabled
  - In other words: if A is almost always* enabled, then A is infinitely often taken

*only finitely many exceptions

Weak vs Strong Fairness

nat x := 0; y := 0
A_x: x := x+1
A_y: even(x) -> y := y+1

- Is the following execution weakly fair to task A_y?

  $(0,0) \text{-}A_x\text{->} (1,0) \text{-}A_x\text{->} (2,0) \text{-}A_x\text{->} (3,0) \ldots \text{-}A_x\text{->} (105,0) \text{-}A_x\text{->} \ldots$
- Task A_y is not continuously enabled and, thus, this execution is weakly fair according to Definition 2 (but it may intuitively seem unfair)
- **Definition 3 (Strong fairness)**: An infinite execution is *strongly fair* to a task A if A is either repeatedly executed or almost always disabled
  - In other words: if A is infinitely often enabled, then A is infinitely often taken
- The above execution is weakly fair to task A_y, but not strongly fair

Fairness Assumption

- Fairness assumption for an asynchronous process P: for each output task and internal task we have either
  - No assumption
  - Weak-fairness assumption
  - Strong-fairness assumption
- Restricts the set of possible infinite executions
- Affects whether the process meets a requirement:
  - Not all executions have to satisfy the given requirement, but all fair executions do

Requirements under Fairness Assumptions

Process P1

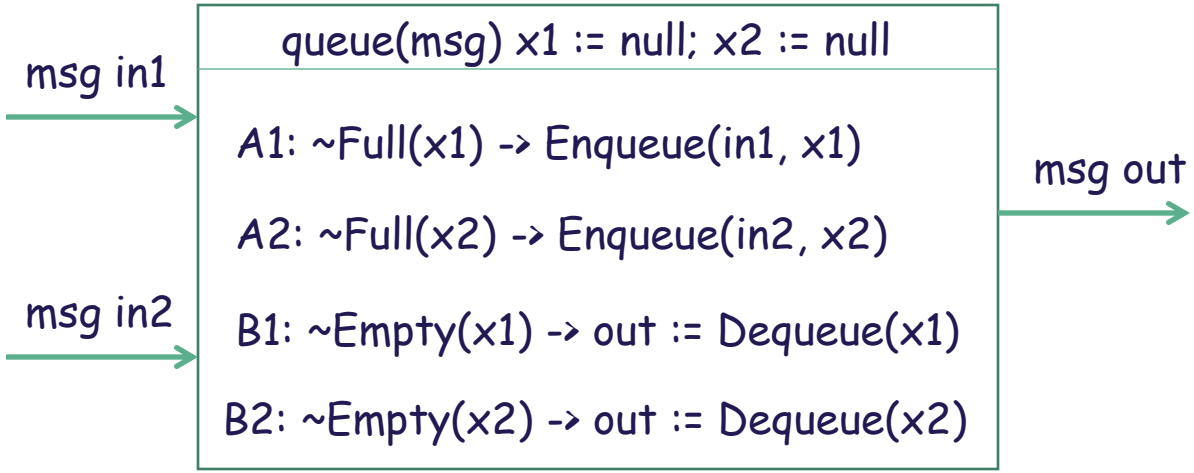
nat x := 0; y := 0
A_x: x := x+1
A_y: y := y+1

Process P3

nat x := 0; y := 0
B_x: x := x+1
B_y: even(x) -> y := y+1

- Under which fairness assumptions do P1 and P3 satisfy the following?
- Requirement: Eventually x+y > 10
  - Both satisfy this without any fairness assumption
- Requirement: Eventually x > 10
  - P1 with weak fairness for A_x, P3 with weak fairness for B_x
- Requirement: Eventually y > 10
  - P1 with weak fairness for A_y, P3 with strong fairness for B_y
- Requirement: Eventually x > y
  - Not satisfied, no matter which fairness assumption we make

Asynchronous Merge

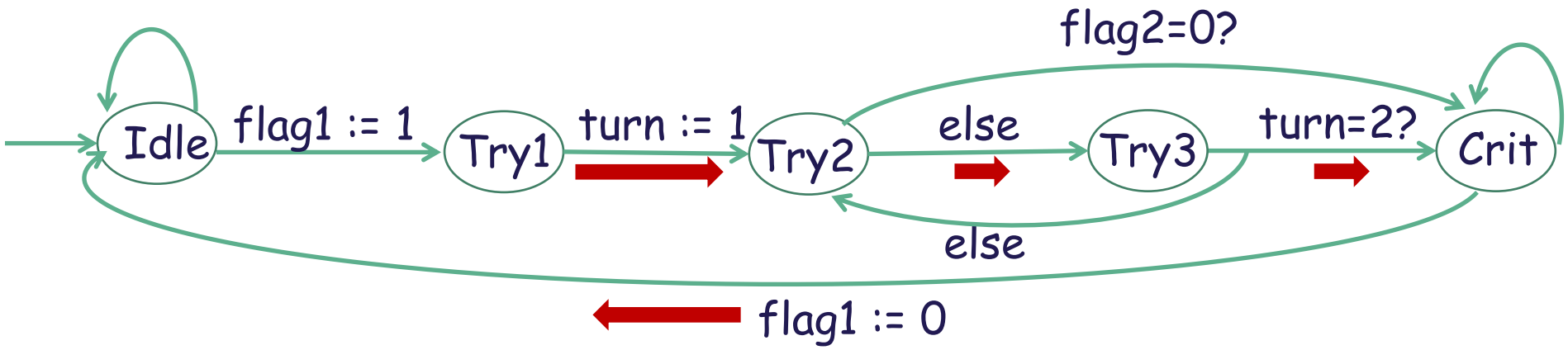


- Consider the following requirement:
  "Whenever an input message is received, it is eventually output"
  - Does not hold without fairness assumptions
- Which fairness assumptions should we make?
  - Weak fairness for output tasks B1 and B2 suffices

Fairness Assumptions for Mutual Exclusion Protocol

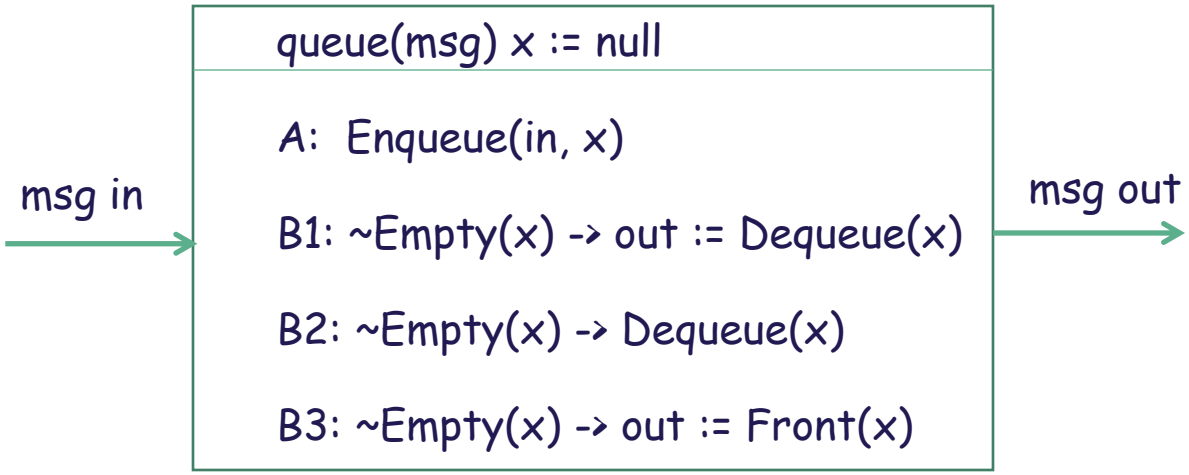
AtomicReg bool flag1 := 0; flag2 := 0; {1,2} turn

Process P1



- Consider the following requirement:
  "Whenever a process wants to enter the critical section, it eventually will"
- Which fairness assumptions should we make?
  - Weak fairness for highlighted steps/tasks suffices

Unreliable FIFO



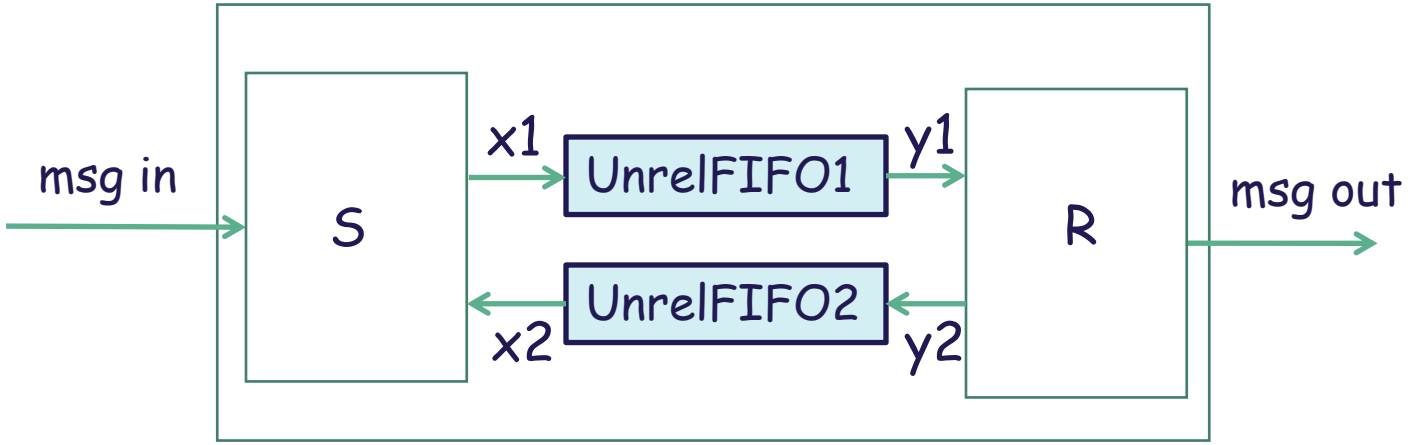
- Models a link that may lose and/or duplicate messages
  - Tasks A and B1 model normal input/output behavior
  - Task B2: Loss of message
  - Task B3: Duplication of message
- What are "natural" fairness assumptions for these tasks?
  - Strong fairness for B1, no assumptions for B2 and B3

Fairness Summary

- Fairness is an assumption about the underlying platform or scheduler
  - The fewer assumptions we make, the better
- For each output and internal task, we can assume weak or strong fairness, as needed
  - Strong fairness is needed if the task can switch between enabled and disabled due to execution of other tasks
- Restricts the set of possible infinite executions, and allows satisfaction of more requirements
  - Does not affect the set of reachable states and safety properties
  - Does not change underlying coordination
- Distinction in terminology:
  - Fairness assumption for tasks (ensure tasks get executed as expected)
  - "Fairness" requirement for protocols (high-level goal)

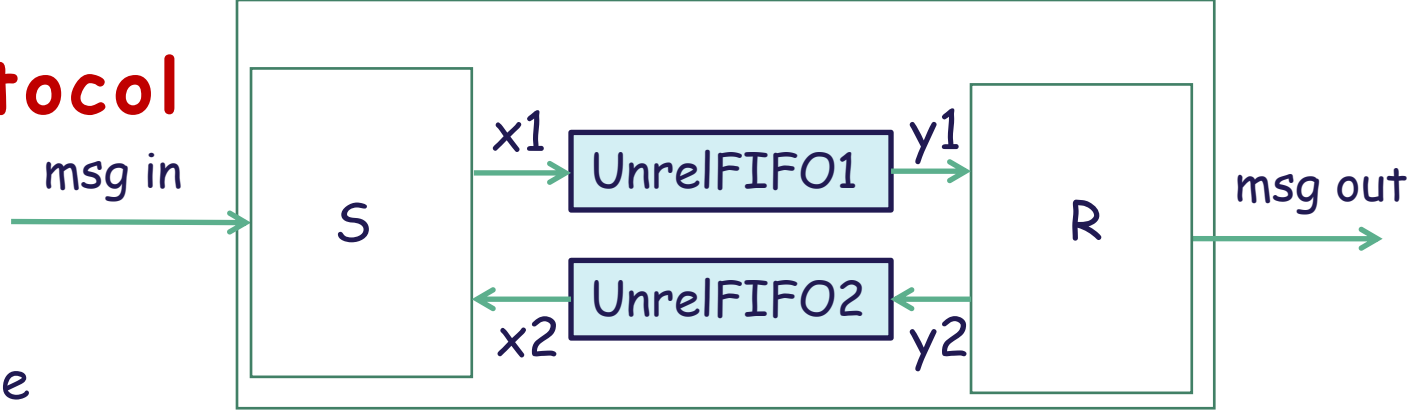
Case Study: Reliable Transmission

Reliable Transmission Problem



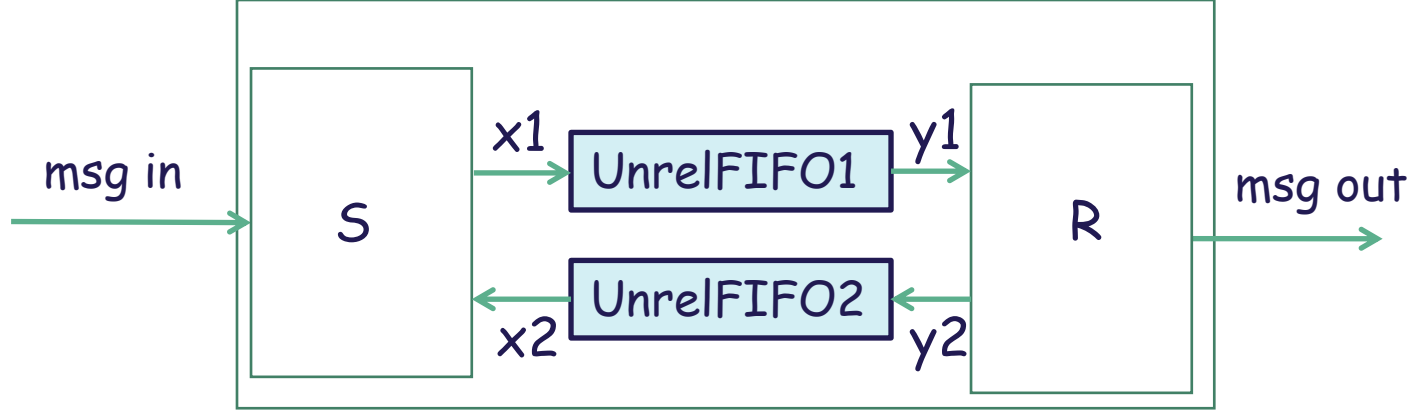
- How to implement a reliable FIFO link using unreliable ones?
- Design asynchronous processes S and R so that the sequence of messages received on the channel in coincides with the sequence of messages delivered on the channel out

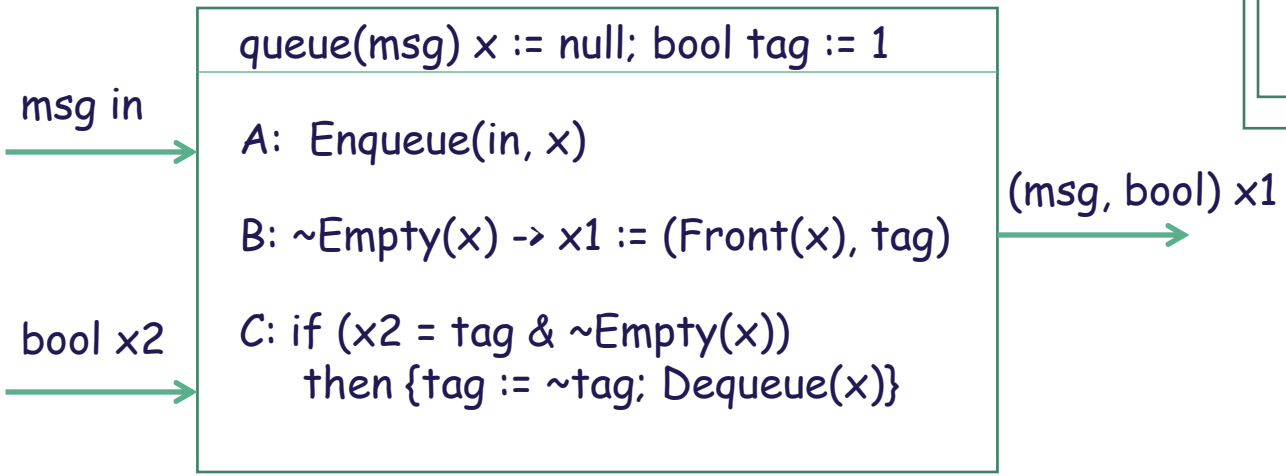
Alternating Bit Protocol



- How can sender S be sure that receiver R got a copy of the message in presence of message losses?
  - S must repeatedly send a message
  - R must send back an acknowledgement, and do so repeatedly
- How can R distinguish between a duplicated/repeated copy and a fresh message?
  - Each message must be tagged with "extra" bits
- Alternating bit protocol:
  - Tagging each message as well as acknowledgement with a single bit suffices
  - Both S and R keep a local tag bit
  - If the tag of the incoming message matches with the local tag, then the message is considered "fresh" and the local tag is toggled

ABP Sender



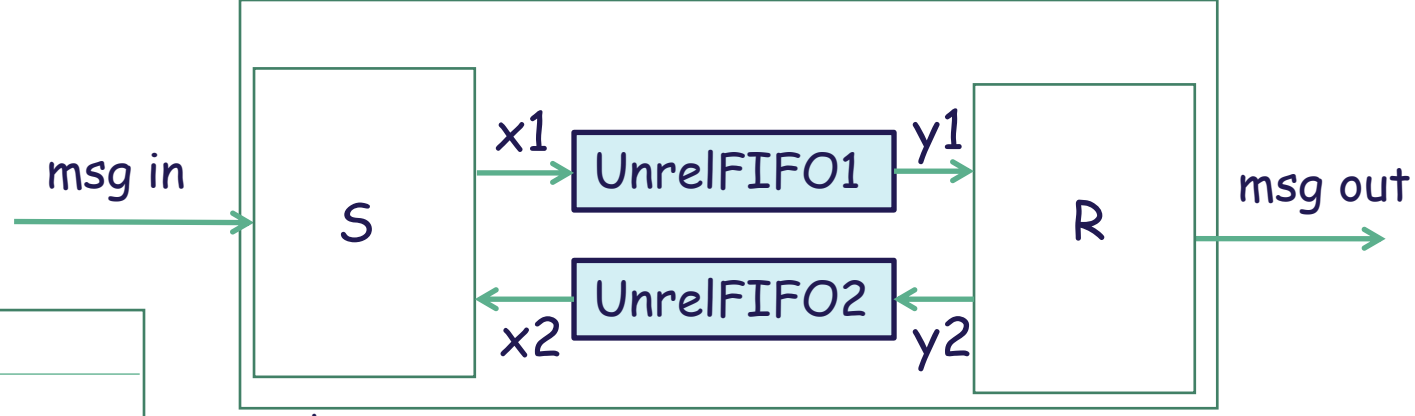


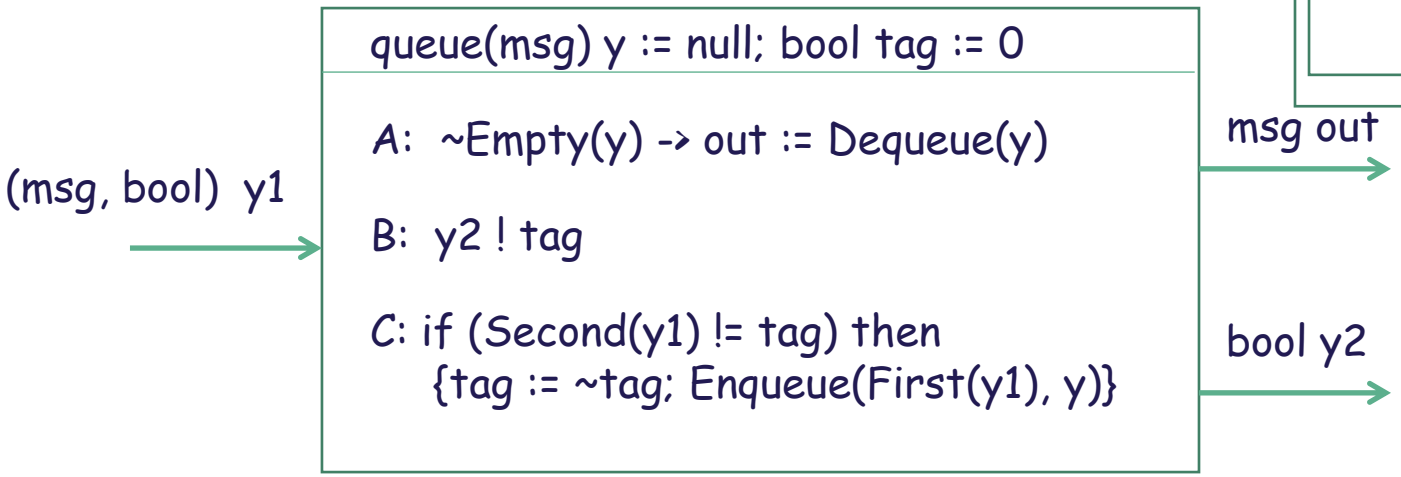
Task A: Store incoming messages in queue x

Task B: Transmit message at front of queue x tagged with current tag
Do not remove the message: this ensures it is transmitted repeatedly

Task C: If acknowledgment matches current tag, the message was successfully delivered; so remove it from x and flip tag

ABP Receiver





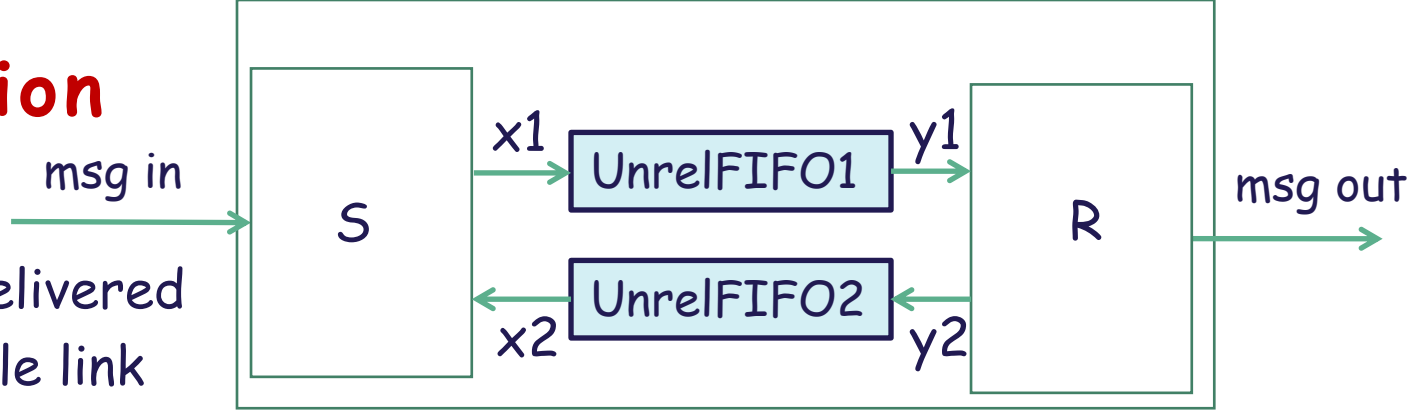
Task A: Transmit outgoing messages from queue y to output channel

Task B: Transmit current tag as acknowledgement on channel y2

Note: Same acknowledgement is potentially transmitted repeatedly

Task C: If tag of incoming message matches own tag, then the message is new; so add it to y and flip tag

ABP Sample Execution



- Initially S.tag = 1 and R.tag = 0
- Suppose S receives message m to be delivered
- S repeatedly sends (m, 1) over unreliable link
- Eventually, R gets at least one copy of (m, 1)
- Before receiving (m, 1), R repeatedly sends acknowledgement 0, which is simply ignored by S
- When R receives (m, 1) the first time, it stores m in queue y (and this message will eventually be transmitted on out), and sets tag to 1
- Duplicate versions of (m, 1) are ignored by R
- R repeatedly sends acknowledgment 1
- Eventually, S receives at least one acknowledgment 1, and then removes m from input queue and sets its tag to 0
- Duplicate versions of acknowledgment 1 are ignored by S
- Messages to be sent are queued up in x, and S will now repeat the whole cycle by sending the next message m' along with tag 0, i.e., (m', 0)